

In [11]:

```
!pip install mediapipe opencv-python
```

Defaulting to user installation because normal site-packages is not writeable
 Requirement already satisfied: mediapipe in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (0.9.0)
 Requirement already satisfied: opencv-python in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (4.6.0.66)
 Requirement already satisfied: attrs>=19.1.0 in c:\programdata\anaconda3\lib\site-packages (from mediapipe) (21.4.0)
 Requirement already satisfied: opencv-contrib-python in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (from mediapipe) (4.6.0.66)
 Requirement already satisfied: flatbuffers>=2.0 in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (from mediapipe) (22.11.23)
 Requirement already satisfied: protobuf<4,>=3.11 in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (from mediapipe) (3.19.6)
 Requirement already satisfied: numpy in c:\programdata\anaconda3\lib\site-packages (from mediapipe) (1.21.5)
 Requirement already satisfied: absl-py in c:\users\bharat sharma\appdata\roaming\python\python39\site-packages (from mediapipe) (1.3.0)
 Requirement already satisfied: matplotlib in c:\programdata\anaconda3\lib\site-packages (from mediapipe) (3.5.2)
 Requirement already satisfied: packaging>=20.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (21.3)
 Requirement already satisfied: pillow>=6.2.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (9.2.0)
 Requirement already satisfied: pyparsing>=2.2.1 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (3.0.9)
 Requirement already satisfied: fonttools>=4.22.0 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (4.25.0)
 Requirement already satisfied: kiwisolver>=1.0.1 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (1.4.2)
 Requirement already satisfied: python-dateutil>=2.7 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (2.8.2)
 Requirement already satisfied: cycler>=0.10 in c:\programdata\anaconda3\lib\site-packages (from matplotlib->mediapipe) (0.11.0)
 Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->mediapipe) (1.16.0)

WARNING: Ignoring invalid distribution -rotobuf (c:\users\bharat sharma\appdata\roaming\python\python39\site-packages)
 WARNING: Ignoring invalid distribution -rotobuf (c:\users\bharat sharma\appdata\roaming\python\python39\site-packages)
 WARNING: Ignoring invalid distribution -rotobuf (c:\users\bharat sharma\appdata\roaming\python\python39\site-packages)
 WARNING: Ignoring invalid distribution -rotobuf (c:\users\bharat sharma\appdata\roaming\python\python39\site-packages)
 WARNING: Ignoring invalid distribution -rotobuf (c:\users\bharat sharma\appdata\roaming\python\python39\site-packages)

Importing necessary libraries

In [44]:

```
import cv2
import math as m
import mediapipe as mp
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

In [45]:

```
mp_drawing = mp.solutions.drawing_utils
mp_holistic = mp.solutions.holistic
```

##Read in Collected Data and Process

In [20]:

```
df = pd.read_csv('coords.csv')
```

In [21]:

```
df.head()
```

Out[21]:

	class	x1	y1	z1	v1	x2	y2	z2	v2	x3	...	z31	v31	x32	y32
0	Standing	0.718350	0.075460	-0.556633	0.999975	0.732501	0.052486	-0.531428	0.999945	0.741256	...	0.490926	0.309952	0.740198	1.129899
1	Standing	0.718421	0.078970	-0.424941	0.999977	0.732563	0.055662	-0.404394	0.999949	0.741254	...	0.323115	0.321531	0.742074	1.141777
2	Standing	0.718312	0.080733	-0.445973	0.999978	0.732214	0.056878	-0.424561	0.999951	0.740735	...	0.323393	0.337261	0.741757	1.149039
3	Standing	0.718029	0.086384	-0.358568	0.999978	0.731237	0.065141	-0.340383	0.999953	0.739484	...	0.302685	0.355475	0.741772	1.146956
4	Standing	0.716747	0.091334	-0.408220	0.999980	0.729226	0.070759	-0.389332	0.999955	0.737131	...	0.314990	0.372577	0.742812	1.133761

5 rows × 133 columns

In [22]:

```
df.tail()
```

Out[22]:

	class	x1	y1	z1	v1	x2	y2	z2	v2	x3	...	z31	v31	x32	y32
2732	Bending	0.640811	0.195959	0.062995	0.997608	0.647675	0.170999	0.025904	0.997666	0.650397	...	0.102459	0.148947	0.770287	1.180391
2733	Bending	0.660366	0.189389	0.063715	0.997652	0.665472	0.164347	0.031510	0.997711	0.668373	...	0.131589	0.144269	0.762383	1.172978
2734	Bending	0.664510	0.181341	0.053599	0.997791	0.671355	0.156402	0.023433	0.997850	0.674247	...	0.154700	0.141438	0.747556	1.168062
2735	Bending	0.667580	0.176877	0.060501	0.997977	0.674071	0.152167	0.029723	0.998031	0.677353	...	0.150948	0.138198	0.753345	1.172244
2736	Bending	0.667323	0.177113	0.051774	0.998121	0.675047	0.151976	0.024309	0.998174	0.678069	...	0.153909	0.137257	0.753651	1.162486

5 rows × 133 columns

In [23]:

```
df.shape
```

Out[23]:

(2737, 133)

In [24]:

```
df.describe()
```

Out[24]:

	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	...	z31	v31	x32	y32
count	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	...	2737.000000	2737.000000	2737.000000	2737.000000
mean	0.601871	0.237682	-0.412736	0.995316	0.617097	0.202183	-0.402316	0.992715	0.626206	0.202229	...	0.196245	0.148947	0.740198	1.129899
std	0.123891	0.114531	0.344906	0.015786	0.125478	0.112284	0.332774	0.027338	0.126547	0.111816	...	0.305475	0.309952	0.740198	1.129899
min	0.005324	-0.172922	-1.790220	0.824996	0.058288	-0.195486	-1.853379	0.754173	0.072995	-0.186643	...	-0.502685	0.309952	0.740198	1.129899
25%	0.528982	0.195826	-0.612189	0.997661	0.542261	0.161995	-0.581387	0.997480	0.552655	0.161188	...	-0.065475	0.309952	0.740198	1.129899
50%	0.593356	0.255180	-0.332774	0.999841	0.615322	0.219474	-0.330337	0.999791	0.624976	0.217484	...	0.220229	0.309952	0.740198	1.129899
75%	0.675936	0.302302	-0.183680	0.999958	0.691495	0.264089	-0.158697	0.999943	0.701853	0.264878	...	0.410926	0.309952	0.740198	1.129899
max	1.015943	0.802218	0.188332	0.999995	1.022597	0.807167	0.135243	0.999987	1.028872	0.805636	...	1.183761	0.309952	0.740198	1.129899

8 rows × 132 columns

In [25]:

```
df['class']
```

Out[25]:

```
0      Standing
1      Standing
2      Standing
3      Standing
4      Standing
...
2732    Bending
2733    Bending
2734    Bending
2735    Bending
2736    Bending
Name: class, Length: 2737, dtype: object
```

In [26]:

```
#Train Custom Model Using Scikit Learn
from sklearn.model_selection import train_test_split
X = df.drop('class', axis=1) # features
y = df['class'] # target value
```

In [27]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1234)
```

In [28]:

```
y_test
```

Out[28]:

```
1175    Sitting_right
1528    Sitting_left
1305    Sitting_left
1832    Standing_up
242      Sitting
...
1730    Sitting_left
1505    Sitting_left
488      Sitting
2126    Standing_down
2549      Bending
Name: class, Length: 822, dtype: object
```

In [29]:

```
#Train Machine Learning Classification Model
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression, RidgeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

In [30]:

```
pipelines = {
    'lr':make_pipeline(StandardScaler(), LogisticRegression()),
    'rc':make_pipeline(StandardScaler(), RidgeClassifier()),
    'rf':make_pipeline(StandardScaler(), RandomForestClassifier()),
    'gb':make_pipeline(StandardScaler(), GradientBoostingClassifier()),
}
```

In [31]:

```
fit_models = {}
for algo, pipeline in pipelines.items():
    model = pipeline.fit(X_train, y_train)
    fit_models[algo] = model
```

In [32]:

```
fit_models
```

Out[32]:

```
{'lr': Pipeline(steps=[('standardscaler', StandardScaler()),
                        ('logisticregression', LogisticRegression())]),
 'rc': Pipeline(steps=[('standardscaler', StandardScaler()),
                        ('ridgeclassifier', RidgeClassifier())]),
 'rf': Pipeline(steps=[('standardscaler', StandardScaler()),
                        ('randomforestclassifier', RandomForestClassifier())]),
 'gb': Pipeline(steps=[('standardscaler', StandardScaler()),
                        ('gradientboostingclassifier', GradientBoostingClassifier())])}
```

In [33]:

```
fit_models['rf'].predict(X_test)
```

Out[33]:

```
array(['Sitting_right', 'Sitting_left', 'Sitting_left', 'Standing_up',
       'Sitting', 'Bending', 'Standing_up', 'Sitting', 'Bending',
       'Standing', 'Standing_down', 'Sitting', 'Standing_up', 'Sitting',
       'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Sitting',
       'Standing', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',
       'Standing_down', 'Standing_down', 'Standing', 'Bending', 'Sitting',
       'Bending', 'Standing_down', 'Sitting', 'Bending', 'Standing_down',
       'Sitting', 'Sitting_left', 'Bending', 'Standing_down', 'Sitting',
       'Sitting_left', 'Sitting', 'Standing_down', 'Bending', 'Sitting',
       'Sitting_left', 'Bending', 'Bending', 'Sitting', 'Sitting',
       'Standing_down', 'Standing_up', 'Sitting', 'Sitting',
       'Sitting_right', 'Sitting_left', 'Bending', 'Sitting', 'Bending',
       'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right',
       'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
       'Standing_down', 'Bending', 'Sitting', 'Sitting_left',
       'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Standing_down',
       'Sitting', 'Sitting', 'Sitting_left', 'Standing_down', 'Bending',
```

In [34]:

```
# Evaluate and Serialize Model
from sklearn.metrics import accuracy_score # Accuracy metrics
import pickle
```

In [35]:

```
for algo, model in fit_models.items():
    yhat = model.predict(X_test)
    print(algo, accuracy_score(y_test, yhat))
```

```
lr 0.9987834549878345
rc 0.9951338199513382
rf 1.0
gb 0.9987834549878345
```

In [36]:

```
fit_models['rf'].predict(X_test)
```

Out[36]:

```
array(['Sitting_right', 'Sitting_left', 'Sitting_left', 'Standing_up',
       'Sitting', 'Bending', 'Standing_up', 'Sitting', 'Bending',
       'Standing', 'Standing_down', 'Sitting', 'Standing_up', 'Sitting',
       'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Sitting',
       'Standing', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',
       'Standing_down', 'Standing_down', 'Standing', 'Bending', 'Sitting',
       'Bending', 'Standing_down', 'Sitting', 'Bending', 'Standing_down',
       'Sitting', 'Sitting_left', 'Bending', 'Standing_down', 'Sitting',
       'Sitting_left', 'Sitting', 'Standing_down', 'Bending', 'Sitting',
       'Sitting_left', 'Bending', 'Bending', 'Sitting', 'Sitting',
       'Standing_down', 'Standing_up', 'Sitting', 'Sitting',
       'Sitting_right', 'Sitting_left', 'Bending', 'Sitting', 'Bending',
       'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right',
       'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
       'Standing_down', 'Bending', 'Sitting', 'Sitting_left',
       'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Standing_down',
       'Sitting', 'Sitting', 'Sitting_left', 'Standing_down', 'Bending',
```

In [37]:

```
y_test
```

Out[37]:

```
1175    Sitting_right
1528    Sitting_left
1305    Sitting_left
1832    Standing_up
242     Sitting
...
1730    Sitting_left
1505    Sitting_left
488     Sitting
2126    Standing_down
2549    Bending
Name: class, Length: 822, dtype: object
```

In [38]:

```
with open('body_language.pkl', 'wb') as f:
    pickle.dump(fit_models['rf'], f)
```

In [39]:

```
with open('body_language.pkl', 'rb') as f:
    model = pickle.load(f)
```

In [40]:

```
model
```

Out[40]:

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                  ('randomforestclassifier', RandomForestClassifier())])
```

In [41]:

```
#Test in Real Time
```


In [42]:

```

cap = cv2.VideoCapture(0)
# Initiate holistic model
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:

    while cap.isOpened():
        ret, frame = cap.read()

        # Recolor Feed
        image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        image.flags.writeable = False

        # Make Detections
        results = holistic.process(image)

        # Recolor image back to BGR for rendering
        image.flags.writeable = True
        image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)

        # 1. Right hand
        mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(80,22,10), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(80,44,121), thickness=2, circle_radius=2)
                                  )

        # 2. Left Hand
        mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(121,22,76), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(121,44,250), thickness=2, circle_radius=2)
                                  )

        # 3. Pose Detections
        mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(245,117,66), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(245,66,230), thickness=2, circle_radius=2)
                                  )

        # Export coordinates
        try:
            # Extract Pose Landmarks
            pose = results.pose_landmarks.landmark
            pose_row = list(np.array([[landmark.x, landmark.y, landmark.z, landmark.visibility] for landmark in pose]).flatten())

            # Create rows
            row = pose_row

            # Make Detections
            X = pd.DataFrame([row])
            body_language_class = model.predict(X)[0]
            body_language_prob = model.predict_proba(X)[0]
            print(body_language_class, body_language_prob)

            # Grab ear coords
            coords = tuple(np.multiply(
                np.array(
                    (results.pose_landmarks.landmark[mp_holistic.PoseLandmark.LEFT_EAR].x,
                     results.pose_landmarks.landmark[mp_holistic.PoseLandmark.LEFT_EAR].y))
                , [640,480]).astype(int))

            cv2.rectangle(image,
                          (coords[0], coords[1]+5),
                          (coords[0]+len(body_language_class)*20, coords[1]-30),
                          (245, 117, 16), -1)
            cv2.putText(image, body_language_class, coords,
                       cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)

            # Get status box
            cv2.rectangle(image, (0,0), (250, 60), (245, 117, 16), -1)

            # Display Class
            cv2.putText(image, 'CLASS'
                        , (95,12), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1, cv2.LINE_AA)
            cv2.putText(image, body_language_class.split(' ')[0]
                        , (90,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)

            # Display Probability
            cv2.putText(image, 'PROB'
                        , (15,12), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1, cv2.LINE_AA)
            cv2.putText(image, str(round(body_language_prob[np.argmax(body_language_prob)],2))
                        , (10,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA)

        except:

```

```
        pass

    cv2.imshow('Raw Webcam Feed', image)

    if cv2.waitKey(10) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```


localhost:8889/notebooks/OneDrive/Desktop/Program folder/Data collection human-Model-Real-time.ipynb

[illegible]

Sitting [0. 0.97 0.01 0. 0. 0.02 0.]
Sitting [0. 0.97 0.01 0. 0. 0.02 0.]
Sitting [0. 0.97 0.01 0. 0. 0.02 0.]
Sitting [0. 0.97 0.01 0. 0. 0.02 0.]

[illegible]

[illegible]

```
Sitting [0. 0.97 0.01 0. 0. 0.02 0. ]
Sitting [0. 0.97 0.01 0. 0. 0.02 0. ]
Sitting [0. 0.97 0.01 0. 0. 0.02 0. ]
```

[illegible]

[illegible]

```
Sitting [0. 0.98 0. 0. 0. 0.02 0. ]
Sitting [0. 0.99 0. 0. 0. 0.01 0. ]
Sitting [0. 0.97 0. 0. 0. 0.01 0.02]
Sitting [0. 0.98 0.01 0. 0. 0.01 0. ]
```

```
Sitting [0. 0.98 0.01 0. 0. 0.01 0. ]
Sitting [0. 0.98 0.01 0. 0. 0.01 0. ]
Sitting [0.01 0.96 0.02 0. 0. 0.01 0. ]
Sitting [0.01 0.93 0.03 0. 0. 0.01 0.02]
Sitting [0.01 0.95 0.02 0. 0. 0.01 0.01]
Sitting [0.01 0.93 0.04 0. 0. 0.01 0.01]
Sitting [0.01 0.92 0.03 0. 0. 0.01 0.03]
Sitting [0.01 0.79 0.1 0.02 0. 0.02 0.06]
Sitting [0.01 0.86 0.09 0. 0. 0.02 0.02]
Sitting [0.01 0.9 0.05 0. 0. 0.02 0.02]
Sitting [0.01 0.82 0.1 0. 0. 0.04 0.03]
Sitting [0.01 0.75 0.14 0. 0. 0.04 0.06]
Sitting [0. 0.49 0.39 0.05 0. 0.02 0.05]
Sitting [0.02 0.55 0.32 0.04 0. 0.02 0.05]
```

